

Project Structure (Presentation-Ready)

1. Purpose

- Local AI coding agent for multi-project development.
- Uses LLM providers (OpenRouter, Hugging Face, Mistral) with safe tool execution.
- Enforces project isolation under workspace/.

2. High-Level Architecture

- main.py: entrypoint.
- openrouter_agent/cli.py: command router and interactive loop.
- openrouter_agent/agents/core.py: plan -> execute -> review/fix runtime.
- openrouter_agent/providers/: model routing, discovery, ranking.
- openrouter_agent/tools/: file/shell operations + tool schemas.
- plugins.py + plugins/: extension system.

3. Core Runtime Flow

- User prompt or slash command.
- Planner generates task steps.
- Executor runs steps with bounded tool iterations.
- Optional reviewer/fixer rounds for autonomous correction.
- Logs/audit/checkpoints persisted per active project.

4. Project Isolation Model

- All project work lives in workspace/<project>/.
- Per-project memory, session, index, command history, and task history.
- Git operations scoped to active project repository.
- Path normalization/safety blocks traversal outside project boundary.

5. Safety & Governance

- Shell command validation and confirmation gates.
- Dry-run mode for mutation preview.
- Edit-scope enforcement in /edit and scoped worker subagents.
- Plugin hook capabilities for policy control.

6. Extensibility

- Plugin manifest: plugins.json.
- Plugin modules can add slash commands and lifecycle hooks.

7. Operational Data Layout

- workspace/: managed projects.
- logs/: per-project audit/history/runs.
- backups/: pre-edit file backups.
- snapshots/: ZIP exports/snapshots.
- SKILL/: guidance packs.

8. Quality & Testing

- tests/ covers CLI, isolation, shell safety, providers, plugins, runtime, checkpoints.
- Run: python -m unittest discover -s tests -v

9. Suggested Slide Deck

- 1) Problem & Goal
- 2) System Overview
- 3) Architecture Diagram
- 4) Runtime Workflow
- 5) Safety & Isolation
- 6) Extensibility via Plugins
- 7) Testing & Reliability